

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ**

Ордена Трудового Красного Знамени

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Московский технический университет связи и информатики»

Кафедра «Математическая Кибернетика и Информационные технологии»

Информационные технологии и программирование

Лабораторная работа №8

Многопоточность

Выполнил: Студент группы

БВТ2402

Юдин Владимир

Москва

2025

Цель работы

Целью лабораторной работы является изучение механизма пользовательских аннотаций в языке Java, а также освоение практического применения Stream API и средств многопоточности из пакета `java.util.concurrent` для обработки данных.

Задачи работы

В ходе выполнения лабораторной работы необходимо:

- создать пользовательскую аннотацию для пометки обработчиков данных;
- реализовать класс управления обработкой данных;
- реализовать несколько классов-обработчиков данных;
- применить Stream API для обработки коллекций;
- реализовать параллельную обработку данных;
- выполнить тестирование приложения с использованием входного файла.

Описание архитектуры приложения

Разработанное приложение представляет собой консольное Java-приложение для обработки текстовых данных, загружаемых из файла.

Архитектура приложения основана на модульном подходе и включает следующие компоненты:

- менеджер данных, управляющий загрузкой, обработкой и сохранением данных;
- интерфейс обработчика данных, определяющий общий контракт;
- набор конкретных обработчиков данных;
- пользовательскую аннотацию для идентификации обработчиков.

Такое разделение позволяет легко добавлять новые обработчики без изменения основной логики программы.

Использование пользовательской аннотации

Для пометки классов, предназначенных для обработки данных, была создана пользовательская аннотация `DataProcessor`.

Аннотация применяется на уровне класса и используется менеджером данных для проверки корректности регистрируемых обработчиков.

Аннотация сохраняется во время выполнения программы, что позволяет выполнять проверку наличия аннотации в runtime.

```
1 package lab_no8;
2 import java.lang.annotation.*;
3
4 @Retention(RetentionPolicy.RUNTIME)
5 @Target(ElementType.TYPE)
6 public @interface DataProcessor {
7 }
8
```

Интерфейс обработчика данных

Для обеспечения единообразия всех обработчиков данных был создан интерфейс обработчика.

Интерфейс определяет метод обработки данных, который принимает коллекцию данных и возвращает результат обработки.

Использование интерфейса позволяет менеджеру данных работать с обработчиками абстрактно, не зная их конкретной реализации.

```
1  package lab_no8;
2  import java.util.List;
3
4  public interface Processor {
5      List<String> process(List<String> data);
6  }
```

Класс управления обработкой данных

Класс менеджера данных отвечает за:

- загрузку данных из входного файла;
- регистрацию обработчиков данных;
- запуск обработки данных в несколько потоков;
- сбор результатов обработки;
- сохранение итоговых данных в выходной файл.

Для реализации параллельной обработки используется пул потоков из пакета `java.util.concurrent`.

Каждый обработчик выполняется в отдельном потоке и работает с копией исходных данных.

```

8
9 public class DataManager {
10
11     private final List<String> data = new ArrayList<>();
12
13     private final List<Processor> processors = new ArrayList<>();
14
15     private final ExecutorService executor = Executors.newFixedThreadPool(nThreads: 4);
16
17     public void registerDataProcessor(Processor p) {
18         if (!p.getClass().isAnnotationPresent(annotationClass: DataProcessor.class)) {
19             throw new IllegalArgumentException("Класс " + p.getClass().getName() + " не помечен @DataProcessor");
20         }
21         processors.add(p);
22     }
23
24     public void loadData(String source) throws IOException {
25         data.clear();
26         data.addAll(Files.readAllLines(Path.of(source)));
27         System.out.println("Загружено строк: " + data.size());
28     }
29
30     public void processData() {
31         if (processors.isEmpty()) {
32             System.out.println("Нет обработчиков!");
33             return;
34         }
35
36         List<Future<List<String>>> results = processors.stream()
37             .map(p -> executor.submit(() -> p.process(new ArrayList<>(data))))
38             .collect(Collectors.toList());
39
40         List<String> combined = new ArrayList<>();
41
42         for (Future<List<String>> f : results) {
43             try {
44                 combined.addAll(f.get());
45             } catch (Exception e) {
46                 e.printStackTrace();
47             }
48         }
49
50         data.clear();
51         data.addAll(combined);
52
53         System.out.println("После обработки строк: " + data.size());
54     }
55
56     public void saveData(String dest) throws IOException {
57         Files.write(Path.of(dest), data);
58         System.out.println("Сохранено в файл: " + dest);
59     }
60 }

```

Обработчики данных

В рамках работы реализовано несколько обработчиков данных:

- обработчик фильтрации данных;
- обработчик преобразования данных;
- обработчик агрегации данных.

Каждый обработчик выполняет одну логическую операцию над данными и не зависит от других обработчиков.

Все обработчики используют Stream API для обработки коллекций данных.

```

1  package lab_no8;
2  import java.util.List;
3  import java.util.stream.Collectors;
4
5  @DataProcessor
6  public class FilterShortLines implements Processor {
7
8      @Override
9      public List<String> process(List<String> data) {
10         return data.stream()
11             .map(String::trim)
12             .filter(s -> s.length() > 3)
13             .collect(Collectors.toList());
14     }
15 }

```

```

1  package lab_no8;
2  import java.util.List;
3
4  @DataProcessor
5  public class SummaryProcessor implements Processor {
6
7      @Override
8      public List<String> process(List<String> data) {
9         long count = data.size();
10        long unique = data.stream().distinct().count();
11
12        return List.of(
13            "SUMMARY",
14            "LINES: " + count,
15            "UNIQUE: " + unique
16        );
17     }
18 }

```

```

1  package lab_no8;
2  import java.util.List;
3  import java.util.stream.Collectors;
4
5  @DataProcessor
6  public class UpperCaseProcessor implements Processor {
7
8      @Override
9      public List<String> process(List<String> data) {
10         return data.stream()
11             .map(String::toUpperCase)
12             .collect(Collectors.toList());
13     }
14 }

```

Использование Stream API

Stream API применяется в обработчиках данных для выполнения операций фильтрации, преобразования и агрегации.

Использование Stream API позволяет реализовать обработку данных в декларативном стиле, повысить читаемость кода и сократить количество вспомогательных конструкций.

Многопоточность

Для параллельного выполнения обработчиков используется пул потоков.

Каждый обработчик запускается как отдельная задача, а результаты их выполнения собираются менеджером данных.

Такой подход повышает производительность обработки и демонстрирует использование средств многопоточности в Java.

Входные и выходные данные

Входные данные представляют собой текстовый файл, содержащий набор строк.

Каждая строка рассматривается как отдельный элемент данных.

Результаты обработки сохраняются в новый текстовый файл после завершения работы всех обработчиков.

```
1  apple
2  dog
3  watermelon
4  hi
5  cat
6  banana
7  kiwi
8  hello world
9  foo
10 bar
11 test
12 123
```

```
1  apple
2  watermelon
3  banana
4  kiwi
5  hello world
6  test
7  APPLE
8  DOG
9  WATERMELON
10 HI
11 CAT
12 BANANA
13 KIWI
14 HELLO WORLD
15 FOO
16 BAR
17 TEST
18 123
19 SUMMARY
20 LINES: 12
21 UNIQUE: 12
22
```


Тестирование приложения

Работа приложения была протестирована на наборе входных данных.

В результате выполнения программы данные были успешно загружены, обработаны несколькими обработчиками и сохранены в выходной файл.

Полученные результаты соответствуют ожидаемым.

```
1  package lab_no8;
2  public class Main {
    Run | Debug
3      public static void main(String[] args) throws Exception {
4
5          DataManager dm = new DataManager();
6
7          dm.registerDataProcessor(new FilterShortLines());
8          dm.registerDataProcessor(new UpperCaseProcessor());
9          dm.registerDataProcessor(new SummaryProcessor());
10
11         dm.loadData(source: "input.txt");
12         dm.processData();
13         dm.saveData(dest: "output.txt");
14
15         dm.shutdown();
16     }
17 }
18
```

Вывод

В ходе выполнения лабораторной работы были изучены механизмы пользовательских аннотаций, Stream API и многопоточности в Java.

Было разработано приложение для обработки данных, корректно выполняющее загрузку, параллельную обработку и сохранение результатов.

Поставленные цели и задачи лабораторной работы были полностью выполнены.